COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Oct 21, 2026	**DATE SUBMITTED:**	Oct 23, 2026
NAME:	Bal, Kevin Ranz J. Braganza, Ralph Angelov F.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14 **Algorithmic Complexity**

1. PROCEDURE OUTPUT

Tabulated below, are a number of functions against n (from 1 to 10)

```
A = (log2 n) {log to base 2 of n}
B = n {linear in n}
C = (* n (log2 n)) {n log n}
D = (* n n) {quadratic in n}
E = (* n n n) {cubic in n}
F = (expt 2 n) {exponential in n}
G = (expt 3 n) {exponential in n}
H = (fact n) {factorial in n}
```

n	A	B	C	D	E	F	G	H
1	0.0	1	0.0	1	1	2	3	1
2	1.0	2	2.0	4	8	4	9	2
3	1.0	3	4.0	9	27	8	27	6
4	2.0	4	8.0	16	64	16	81	24
5	2.0	5	11.0	25	125	32	243	120
6	2.0	6	15.0	36	216	64	729	720
7	2.0	7	19.0	49	343	128	2187	5040
8	3.0	8	24.0	64	512	256	6561	40320
9	3.0	9	28.0	81	729	512	19683	362880
10	3.0	10	33.0	100	1000	1024	59049	3628800

Think of this as algorithms A through H with complexities as defined above, showing growth rate versus input size n. Tabulated below are functions F, G and H from above (ie 2 to power n, 3 to power n, and n factorial). Problem size n varies from 10 to 100 in steps of 10. I have assumed that we have a machine that can perform "the principle activity of the algorithm" in a micro second (ie. if we are considering a graph colouring algorithm, it can compare the colour of two nodes in a millionth of a second). The columns give the number of years this machine would take to execute those algorithms on problems of size n (note: YEARS). This is expressed as 10^x, "10 raised to the power x".

n	F	G	H
10	10^-10	10^-8	10^-6
20	10^-7	10^-3	10^4
30	10^-4	10^0	10^18
40	10^-1	10^5	10^34
50	10^1	10^10	10^50
60	10^4	10^15	10^68
70	10^7	10^19	10^86
80	10^10	10^24	10^105
90	10^13	10^29	10^124
100	10^16	10^34	10^144

Therefore, if we have a problem of size (lets say 40) and the machine specified above, if the best algorithm is O(2**n) it will take 1 year, if the best algorithm is O(3**n) it will take 100,000 years, and if the best algorithm is O(n!) it will take

10,000,000,000,000,000,000,000,000,000,000,000 years

approximately. Out of interest, the age of the universe is estimated to be between 15 and 20 billion years old, i.e. 20,000,000,000 years. That is, even at modest values of n we are presented with problems that will never be solved. It is almost tempting to say that from a computational perspective, the universe is a small thing.

Contrast and compare two algorithms used in sorting. For the two algorithms, identify the best and worst case. Additionally, provide an analysis of the algorithms if they were to sort data with the following values of n: 100, 1,000, and 10,000 (use experimental and theoretical analysis tools used in previous activities).

Algorithm	Best Case	Worst Case	Analysis
Merge Sort	O(n log n)	O(n log n)	This sorting algorithm works by splitting the list/array in half/midpoint multiple times until each of the values in the array/list is single and then comparing it from lowest to highest and merging it again until it returns to become a list or array but the values inside of it was already in correct position/sorted. It takes almost the exact same time no matter how messy the list/array is.This still gives both worst and best case scenario of time complexity of O(n log n), this gives a reason of faster and dependable than bubble sort when handling large amount of list/data

Bubble Sort	O(n)	O(n^2)	This sorting algorithm operates by looking at the pair of numbers/values next to each other and swapping them if they're in the wrong order of list/data which repeats until the whole list of number/values are on the right place/already sorted.Its best-case happens when the list is already sorted that giving it a time complexity of O(n), because it only needs to check the list/array in one time. While the worst-case happens when the list is completely backward, resulting in a slow time complexity of O(n^2) because every single pair must be swapped over and over.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

Cases	Merge Sort	Bubble Sort
Best Case	============================== | BEST CASE | ==== Input Size (N): 100 BEST CASE (Sorted): 39 microseconds Input Size (N): 1000 BEST CASE (Sorted): 402 microseconds Input Size (N): 10000 BEST CASE (Sorted): 4432 microseconds	============================== | BEST CASE | == Input Size (N): 100 BEST CASE (Sorted): 9 microseconds Input Size (N): 1000 BEST CASE (Sorted): 350 microseconds Input Size (N): 10000 BEST CASE (Sorted): 31030 microseconds
Worst Case	============================== | WORST CASE | ==== Input Size (N): 100 WORST CASE (Reverse Sorted): 38 microseconds Input Size (N): 1000 WORST CASE (Reverse Sorted): 557 microseconds Input Size (N): 10000 WORST CASE (Reverse Sorted): 5608 microseconds	============================== | WORST CASE | ===== Input Size (N): 100 WORST CASE (Reverse Sorted): 11 microseconds Input Size (N): 1000 WORST CASE (Reverse Sorted): 1051 microseconds Input Size (N): 10000 WORST CASE (Reverse Sorted): 102136 microseconds

Analysis [Bal]

As shown in the console output showing the time for each algorithm to sort an array in best and worst case scenarios, bubble sort shows a large increase in time as the input size grows from 100 to 1,000 and then 10,000. In both its best and worst cases, when the size goes from 1,000 to 10,000, the execution time for bubble sort increases by around 100 times due to the growth in the number of elements. Merge sort shows a consistent execution time for both case scenarios, where the numbers stay relatively similar to each other in both the best and worst conditions. This demonstrates the consistency of the merge sort algorithm. Comparing the two algorithms, bubble sort is faster with a small number of elements like 100, but it becomes significantly slower than merge sort as the number of elements grows to 10,000.

Braganza's Analysis
After running the code we gathered the experimental results, it is clear from the results that Merge Sort works faster than Bubble Sort with increasing input size. With regard to Merge Sort, the execution time increased from 39 microseconds with 100 elements to 4,432 microseconds for 10,000 elements in the sorted data case. The execution time in the reverse sorted case was 38 to 5,608 microseconds. This indicates the efficiency of Merge Sort with larger input since its time complexity is (O(n log n)). The values of execution times in sorted and reverse sorted cases are closer, which is expected because Merge Sort splits the array anyway. In the case of Bubble Sort, the run time for 100 elements is 9 microseconds and 31,030 microseconds for 10,000 elements when the list is sorted. On the other hand for reverse-sorted data, it took 11 microseconds for 100 elements and 102,136 microseconds for 10,000 elements. Thus, it is clear that the Bubble Sort algorithm is highly inefficient when it deals with a large input as the time complexity of Bubble Sort is O(n2). Reverse-sorted data took more time for 10,000 elements than the sorted one. However, as per the given code of Bubble Sort, the number of comparisons remains the same. Therefore, the Bubble Sort algorithm is efficient for small amounts of data as seen with 100 elements while Merge Sort is efficient for large amounts of data.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis.

- For the theoretical analysis, the deliverable must be:
- Pseudocode Analysis
- Time and Space Complexities

For the empirical analysis, the deliverable must be the filled-out table and your analysis

Searching Technique: Linear Search

Code

sortingAlgo.h

```
// Linear Search
int search(std::vector<int> &arr, int target) {
    int n = arr.size();

    for (int i = 0; i < n; i++)
        if (arr[i] == target)
            return i;
    return -1;
}
```

```
// Worst-Case Linear Search
void testLinearSearch(int SIZE) {
    std::vector<int> dataset(SIZE);

    for (int i = 0; i < SIZE; i++)
        dataset[i] = i;

    int choice = SIZE;
    const int REPEAT = 10000;
    auto start = std::chrono::high_resolution_clock::now();

    for (int i = 0; i < REPEAT; i++)
        search(dataset, choice);
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::micro> duration = end - start;
    std::cout << "\nInput Size (N): " << SIZE;
    std::cout << "\nElapsed Time (Average): " << duration.count() / REPEAT << " microseconds\n";
}
```

main.cpp

```
    std::cout << "\n================ | WORST-CASE LINEAR SEARCH | ================\n";

    testLinearSearch(1000);
    testLinearSearch(10000);
    testLinearSearch(100000);

    return 0;
}

```

Pseudocode

```
LinearSearch(A, target)
    FOR each element in A
        IF element = target
            RETURN index
    RETURN -1
```

Pseudocode Analysis [Bal]	As shown in the linear search pseudocode, the algorithm operates with a time complexity of O(n) in the worst-case scenario. This happens because the algorithm uses a loop to check every single element in the list/array one by one. If the target element is at the end of the list or not in the array at all, the loop must run exactly n times, meaning the execution time grows directly alongside the size of the list. For the space complexity, the algorithm uses O(1) space because it executes the search directly on the existing array without allocating any extra memory or creating new space to store data for it .
Pseudocode Analysis [Braganza]	Linear Search checks each element in the array one by one until the target is found. If the target is not present, it continues checking every element before returning -1. The target is set to SIZE, which is the test values that will be used for the activity we are performing (1000, 10000,100000), and it is outside the array's range, so every element is checked.

Searching Technique: Binary Search

Code

sortingAlgo.h

```cpp
// Binary Search
int binarySearch(std::vector<int> &arr, int target, int low, int high) {
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target)
            return mid;
        if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}
```

```cpp
// Worst-Case Binary Search
void testBinarySearch(int SIZE) {
    std::vector<int> dataset(SIZE);

    for (int i = 0; i < SIZE; i++)
        dataset[i] = i;

    int choice = SIZE;
    const int REPEAT = 10000;
    auto start = std::chrono::high_resolution_clock::now();

    for (int i = 0; i < REPEAT; i++)
        binarySearch(dataset, choice, 0, SIZE - 1);
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::micro> duration = end - start;\
    std::cout << "\nInput Size (N): " << SIZE;
    std::cout << "\nElapsed Time (Average): " << duration.count() / REPEAT << " microseconds\n";
}
```

main.cpp

```cpp
int main() {

    std::cout << "\n================ | WORST-CASE BINARY SEARCH | ================\n";

    testBinarySearch(1000);
    testBinarySearch(10000);
    testBinarySearch(100000);
```

Pseudocode

```
BinarySearch(A, target, low, high)
   WHILE low <= high
      mid = low + (high - low) / 2

      IF A[mid] = target
         RETURN mid
      ELSE IF A[mid] < target
         low = mid + 1
      ELSE
         high = mid - 1

   RETURN -1
```

Pseudocode Analysis [Bal]

In this binary search, the algorithm operates with a time complexity of O(log n) in the worst-case. This happens because the algorithm uses a while loop that continuously finds the middle index of a sorted array and compares it to the target

	value. During each process, the algorithm updates either the low or high markers to cut the remaining search area in half from the search list. This constant division allows the algorithm to find the target using very few steps even when the array size gets much larger. For space complexity, it uses O(1) memory because it only creates a few simple variables to track the bounds of the array.
Pseudocode Analysis [Braganza]	Binary Search works on a sorted array by comparing the target with the middle element. If the target is greater than the middle element, it searches the right half; otherwise, it searches the left half. This continues until the target is found or the search range becomes empty. Since we are testing the worst-case scenario, the target is not present in the array. For example, if SIZE is set to 1000, one of our test values, the array contains numbers from 0 to 999. Therefore, the algorithm will not find 1000 and will repeatedly reduce the search range until it terminates.

Time and Space Complexities

Complexities	Linear	Binary
Time Complexities	Best Case: O(1) Worst Case: O(n)	Best Case: O(1) Worst Case: O(log n)
Space Complexities	O(1)	O(1)

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	Input Size (N): 1000 Elapsed Time (Average): 0.04328 microseconds	Input Size (N): 1000 Elapsed Time (Average): 1.5129 microseconds
10,000	Input Size (N): 10000 Elapsed Time (Average): 0.05049 microseconds	Input Size (N): 10000 Elapsed Time (Average): 15.3123 microseconds
100,000	Input Size (N): 100000 Elapsed Time (Average): 0.10629 microseconds	Input Size (N): 100000 Elapsed Time (Average): 156.212 microseconds

Analysis Bal

As shown in the console output showing the time for each algorithm to search an array in worst case scenarios, linear search shows a large increase in time as the list size grows from 1,000 to 10,000 and then 100,000. When the size increases by 10 times, the execution time for linear search also increases about 10 times, giving and displaying a 1.5129 microseconds at 1,000 elements, to 15.3123 microseconds at 10,000 elements, and then to 156.212 microseconds at 100,000 elements to the console. This happens because it has an O(n) time complexity. Based on these observations, the execution time for linear search scales directly with the number of elements because it has to look at every single item in the worst case. While in the binary search console, the output showing the time for each algorithm to search an array in worst case scenarios, binary search shows far more consistent execution time across all sizes due to its O(log n) time complexity. It takes 0.04328 microseconds at 1,000 elements, 0.05049 microseconds at 10,000 elements, and slowly rises to 0.10629 microseconds when reaching 100,000 elements. Based on these observations, binary search is faster than linear search at every phase and its growth in execution time stays low even when the number of elements grows large.

Analysis Braganza

Once we ran our code implementation, it is observed that Binary Search outperforms Linear Search in terms of time taken to execute for all three test cases (1000,10000,100000). The Binary Search execution time rises very slightly **from 0.04328 microseconds for 1,000 elements to 0.10629 microseconds for 100,000 elements** after 10000 repeats. This happens because the Binary Search algorithm splits the sorted array into halves at each step, eliminating many of the elements on each comparison it does.

On the other hand, Linear Search **takes 1.5129 microseconds to process 1,000 elements, 15.3123 microseconds to process 10,000 elements, and finally 156.212 microseconds to process 100,000 elements**. This algorithm's processing time increases with an increase in the number of elements since it processes the array element-by-element until it searches the required target. But since the target is not present in the array, Linear Search must examine all the elements before stopping.

Conclusion Bal

This activity demonstrates and illustrates that the theoretical analyses of sorting and searching algorithms determine the experimental performance, which shows how the theoretical analysis affects the execution speed when the data or input size grows larger. It became clear to me that those sorting algorithms with higher theoretical time complexities like the bubble sort and linear sort as the given example will display a slow run time when handling a larger list size, while the binary and merge sort remain efficient and consistent because their logarithmic and linearithmic models are designed to constantly divide the operations in half, and also shows that each algorithm has their own strengths and weaknesses, including their implementations. With the help of this demonstration of these concepts, it provides a good practice and learnings of how the provided algorithm works, behaves, including their speed of execution with the help of STL chrono to see its runtime.

Conclusion Braganza

In this hands-on activity, I got to know more about Linear Search and Binary Search since initially I did not know

much about these algorithms because I didn't really have the time to immerse myself to learn about them due to time constraints. I learned how to write pseudocode, test different input sizes, and use chrono to measure execution times. We also implemented a repeater in our code so that it will repeat the searches 10,000 times and calculate the average time to make the results more reliable. During the testing, I learned that using a target that was not present in the array allowed us to test the worst-case performance of both algorithms. The results showed that Binary Search was much faster than Linear Search, especially as the input size increased. I also experienced some trial and error while checking the code and making sure the tests measured the algorithms correctly but even though it checks it 10,000 times there are still some instances that searching through 1,000 elements will be slower than searching through 100,000 for some reason.

All in all, I believe that I performed quite well on this activity as I managed to implement both algorithms, vary the size of the inputs, and make comparisons for the actual results with their theoretical complexities. As in the previous activities where we discussed Bubble Sort and Merge Sort, I understood how to analyze the algorithms through a combination of theoretical explanations with actual execution times using the chrono once again. It was quite refreshing after finishing the main.cpp file logic, all I had to do was just copy and paste the same logic to the other algorithm because all I had to do was change what function it calls. Nevertheless, I need to continue studying algorithms and how to interpret the differences in execution time. Also, I need some additional practice in writing pseudocode and understanding its impact on the performance of an algorithm to develop my logical skills even further to make efficient code implementations.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Google (Overview), & QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, Grammar correction, & Idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used for explaining some certain concepts and including clarification in the most interpreted way, fixing grammatical errors when explaining our analysis, and generating an example for the implementation coming from scratch that was not aligned to the activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.